Task 11

```
matti_avishya@LAPTOP-AUU62HPJ:~/task11_pdf$ nano pdf_open.c
matti_avishya@LAPTOP-AUU62HPJ:~/task11_pdf$ ls
pdf_open.c
matti_avishya@LAPTOP-AUU62HPJ:~/task11_pdf$ cp pdf_open.c sample.pdf
matti_avishya@LAPTOP-AUU62HPJ:~/task11_pdf$ ls -l sample.pdf
-rw-r--r-- 1 matti_avishya matti_avishya 970 Aug 14 05:06 sample.pdf
matti_avishya@LAPTOP-AUU62HPJ:~/task11_pdf$ ls
pdf_open.c  sample.pdf
matti_avishya@LAPTOP-AUU62HPJ:~/task11_pdf$ gcc pdf_open.c -o pdf_open
matti_avishya@LAPTOP-AUU62HPJ:~/task11_pdf$ ./pdf_open
Browser process started.
Browser PID: 768

Parent/browser process continues running.
Parent PID: 768
Child/PDF viewer PID: 769

Child process created.
Child PID: 769
Opening PDF using xdg-open...
exec failed: No such file or directory

PDF viewer process finished.
Browser process continues.
matti_avishya@LAPTOP-AUU62HPJ:~/task11_pdf$ ps -ef | grep pdf_open
matti_a+     780     334  0 05:07 pts/0    00:00:00 grep --color=auto pdf_open
matti_avishya@LAPTOP-AUU62HPJ:~/task11_pdf$
```

Task 12

```
matti_avishya@LAPTOP-AUU62HPJ:~$ mkdir task12
matti_avishya@LAPTOP-AUU62HPJ:~$ cd task12
matti_avishya@LAPTOP-AUU62HPJ:~/task12$ nano shell_loop.c
matti_avishya@LAPTOP-AUU62HPJ:~/task12$ ls
shell_loop.c
matti_avishya@LAPTOP-AUU62HPJ:~/task12$ shell_loop.c
shell_loop.c: command not found
matti_avishya@LAPTOP-AUU62HPJ:~/task12$ gcc shell_loop.c -o shell_loop
matti_avishya@LAPTOP-AUU62HPJ:~/task12$ ls
shell_loop  shell_loop.c
matti_avishya@LAPTOP-AUU62HPJ:~/task12$ ./shell_loop
Simple Linux Command Loop
Type 'exit' to quit.

myshell> hello
You entered: hello
myshell> Linux
You entered: Linux
myshell> test
You entered: test
myshell> exit
Exiting shell...
matti_avishya@LAPTOP-AUU62HPJ:~/task12$ gcc shell_loop.c -o shell_loop
matti_avishya@LAPTOP-AUU62HPJ:~/task12$ ./shell_loop
Simple Linux Command Loop
Type 'exit' to quit.

myshell> hello
You entered: hello
myshell> exit
Exiting shell...
matti_avishya@LAPTOP-AUU62HPJ:~/task12$
```

Task 13

```
matti_avishya@LAPTOP-AUU62HPJ:~$ cd ~
matti_avishya@LAPTOP-AUU62HPJ:~$ mkdir -p task13
cd task13
matti_avishya@LAPTOP-AUU62HPJ:~/task13$
matti_avishya@LAPTOP-AUU62HPJ:~/task13$ cat > task13.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 100

/* Process the entered command */
void processCommand(char *command) {
    if (strcmp(command, "hello") == 0) {
        printf("Hello! Welcome to MyShell.\n");
    }
    else if (strcmp(command, "help") == 0) {
        printf("\nAvailable commands:\n");
        printf("  hello - Display greeting\n");
        printf("  help  - Display available commands\n");
        printf("  exit  - Exit the shell\n");
    }
    else if (strlen(command) == 0) {
        /* Do nothing for empty input */
    }
    else {
        printf("Unknown command: %s\n", command);
    }
}

int main() {
    char buffer[MAX_INPUT];

    while (1) {
        printf("myshell> ");
```

```
        printf("myshell> ");
        fflush(stdout);

        fgets(buffer, sizeof(buffer), stdin);

        /* Remove newline */
        buffer[strcspn(buffer, "\n")] = '\0';

        /* Check exit condition */
        if (strcmp(buffer, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }
        else {
            processCommand(buffer);
            fflush(stdout);
        }
    }

    return 0;
}
EOF
matti_avishya@LAPTOP-AUU62HPJ:~/task13$ gcc -Wall -Wextra task13.c -o task13
matti_avishya@LAPTOP-AUU62HPJ:~/task13$ ./task13
myshell> hello
Hello! Welcome to MyShell.
myshell> help

Available commands:
  hello - Display greeting
  help  - Display available commands
  exit  - Exit the shell
myshell> test
Unknown command: test
myshell> exit
Exiting shell...
```

Task 14

```
matti_avishya@LAPTOP-AUU62HPJ:~$ cd ~
matti_avishya@LAPTOP-AUU62HPJ:~$ mkdir -p task14
matti_avishya@LAPTOP-AUU62HPJ:~$ cd task14
matti_avishya@LAPTOP-AUU62HPJ:~/task14$
matti_avishya@LAPTOP-AUU62HPJ:~/task14$ cat > task14.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>

#define MAX_INPUT 100
#define MAX_HISTORY 100

char history[MAX_HISTORY][MAX_INPUT];
int history_count = 0;
int history_pos = 0;

struct termios original_termios;

/* Disable raw mode and restore terminal */
void disableRawMode(void)
{
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &original_termios);
}

/* Enable non-canonical mode */
void enableRawMode(void)
{
    struct termios raw;

    tcgetattr(STDIN_FILENO, &original_termios);
    atexit(disableRawMode);

    raw = original_termios;

    raw.c_lflag &= ~(ECHO | ICANON);
    raw.c_cc[VMIN] = 1;
    raw.c_cc[VTIME] = 0;

    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}
```

```
void addHistory(const char *command)
{
    if (strlen(command) == 0)
        return;

    if (history_count < MAX_HISTORY)
    {
        strcpy(history[history_count], command);
        history_count++;
    }
    else
    {
        for (int i = 1; i < MAX_HISTORY; i++)
            strcpy(history[i - 1], history[i]);

        strcpy(history[MAX_HISTORY - 1], command);
    }

    history_pos = history_count;
}

/* Clear current input line */
void clearLine(int length)
{
    for (int i = 0; i < length; i++)
        printf("\b \b");

    fflush(stdout);
}

/* Display history */
void showHistory(void)
{
    printf("\n");

    for (int i = 0; i < history_count; i++)
        printf("%d  %s\n", i + 1, history[i]);
}

/* Read command with arrow-key history */
void readCommand(char *buffer)
{
    int length = 0;
    int cursor = 0;
```

```
    char c;

    history_pos = history_count;

    while (1)
    {
        if (read(STDIN_FILENO, &c, 1) != 1)
            continue;

        /* Enter key */
        if (c == '\n' || c == '\r')
        {
            buffer[length] = '\0';
            printf("\n");
            return;
        }

        /* Backspace */
        if (c == 127 || c == 8)
        {
            if (cursor > 0)
            {
                for (int i = cursor; i < length; i++)
                    buffer[i - 1] = buffer[i];

                length--;
                cursor--;

                printf("\b");

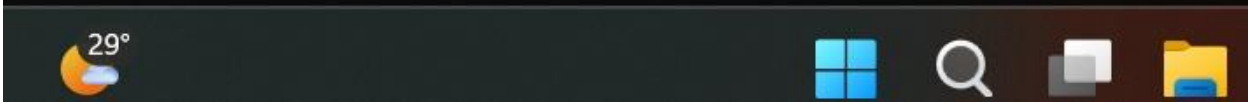
                for (int i = cursor; i < length; i++)
                    printf("%c", buffer[i]);

                printf(" ");

                for (int i = cursor; i <= length; i++)
                    printf("\b");

                fflush(stdout);
            }

            continue;
        }
```



```
    {
        char seq[2];

        if (read(STDIN_FILENO, &seq[0], 1) != 1)
            continue;

        if (read(STDIN_FILENO, &seq[1], 1) != 1)
            continue;

        /* Up Arrow */
        if (seq[0] == '[' && seq[1] == 'A')
        {
            if (history_pos > 0)
            {
                history_pos--;

                clearLine(length);

                strcpy(buffer, history[history_pos]);
                length = strlen(buffer);
                cursor = length;

                printf("%s", buffer);
                fflush(stdout);
            }
        }

        /* Down Arrow */
        else if (seq[0] == '[' && seq[1] == 'B')
        {
            if (history_pos < history_count - 1)
            {
                history_pos++;

                clearLine(length);

                strcpy(buffer, history[history_pos]);
                length = strlen(buffer);
                cursor = length;

                printf("%s", buffer);
                fflush(stdout);
            }
            else if (history_pos == history_count - 1)
```

```
                {
                    history_pos = history_count;

                    clearLine(length);

                    buffer[0] = '\0';
                    length = 0;
                    cursor = 0;

                    fflush(stdout);
                }
            }

            continue;
        }

        /* Normal character */
        if (c >= 32 && c <= 126)
        {
            if (length < MAX_INPUT - 1)
            {
                for (int i = length; i > cursor; i--)
                    buffer[i] = buffer[i - 1];

                buffer[cursor] = c;
                length++;
                cursor++;

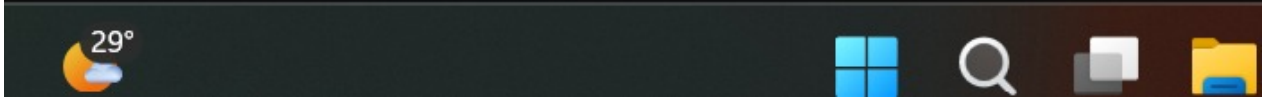
                printf("%s", buffer + cursor - 1);

                for (int i = cursor; i < length; i++)
                    printf("\b");

                fflush(stdout);
            }
        }
    }
}

int main(void)
{
    char command[MAX_INPUT];

    enableRawMode();
```



```
        {
            showHistory();
            continue;
        }

        if (strlen(command) > 0)
        {
            addHistory(command);
            printf("You entered: %s\n", command);
        }
    }

    return 0;
}
EOF
matti_avishya@LAPTOP-AUU62HPJ:~/task14$ gcc -Wall -Wextra task14.c -o task14
matti_avishya@LAPTOP-AUU62HPJ:~/task14$ ./task14
Simple Shell with Command History
Use Up/Down arrows for history.
Type 'history' to display history.
Type 'exit' to quit.

myshell> hello
You entered: hhello
myshell> whellooellorllollodo
You entered: world
myshell> world
You entered: world
myshell> world
You entered: world
myshell> world
You entered: world
myshell> ~orld
You entered: ~
myshell> hirldsldtdory

1  hhello
2  world
3  world
4  world
5  world
6  ~
myshell> eistoryxstoryitorytory
Exiting shell...
```

Task 15

```
matti_avishya@LAPTOP-AUU62HPJ:~/task14$ cd ~
matti_avishya@LAPTOP-AUU62HPJ:~$ mkdir -p task15
matti_avishya@LAPTOP-AUU62HPJ:~$ cd task15
matti_avishya@LAPTOP-AUU62HPJ:~/task15$
matti_avishya@LAPTOP-AUU62HPJ:~/task15$ cat > task15.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pid_t producer, consumer;

    /* Create pipe */
    if (pipe(fd) == -1)
    {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    /* Create producer process */
    producer = fork();

    if (producer == -1)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (producer == 0)
    {
        /* Producer: ls -l */

        close(fd[0]);

        /* Redirect stdout to pipe */
        dup2(fd[1], STDOUT_FILENO);

        close(fd[1]);

        /* Execute ls -l */
```

```
        exit(EXIT_FAILURE);
    }

    /* Create consumer process */
    consumer = fork();

    if (consumer == -1)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (consumer == 0)
    {
        /* Consumer: grep ".c" */

        close(fd[1]);

        /* Redirect stdin from pipe */
        dup2(fd[0], STDIN_FILENO);

        close(fd[0]);

        /* Execute grep ".c" */
        execlp("grep", "grep", ".c", (char *)NULL);

        perror("execlp grep");
        exit(EXIT_FAILURE);
    }

    /* Parent closes both pipe ends */
    close(fd[0]);
    close(fd[1]);

    /* Wait for both children */
    waitpid(producer, NULL, 0);
    waitpid(consumer, NULL, 0);

    printf("Pipeline execution completed.\n");

    return 0;
}
EOF
matti_avishya@LAPTOP-AUU62HPJ:~/task15$ gcc -Wall -Wextra task15.c -o task15
```

```
matti_avishya@LAPTOP-AUU62HPJ:~/task15$ ls
task15  task15.c
matti_avishya@LAPTOP-AUU62HPJ:~/task15$ touch test1.c test2.c test3.txt
matti_avishya@LAPTOP-AUU62HPJ:~/task15$ ls -l
total 20
-rwxr-xr-x 1 matti_avishya matti_avishya 16344 Sep  9 05:13 task15
-rw-r--r-- 1 matti_avishya matti_avishya  1465 Sep  9 05:13 task15.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 test1.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 test2.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 test3.txt
matti_avishya@LAPTOP-AUU62HPJ:~/task15$ ./task15
-rw-r--r-- 1 matti_avishya matti_avishya  1465 Sep  9 05:13 task15.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 test1.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 test2.c
Pipeline execution completed.
matti_avishya@LAPTOP-AUU62HPJ:~/task15$ touch hello.c world.c notes.txt
matti_avishya@LAPTOP-AUU62HPJ:~/task15$ ./task15
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 hello.c
-rw-r--r-- 1 matti_avishya matti_avishya  1465 Sep  9 05:13 task15.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 test1.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 test2.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 world.c
Pipeline execution completed.
matti_avishya@LAPTOP-AUU62HPJ:~/task15$ ls -l | grep ".c"
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 hello.c
-rw-r--r-- 1 matti_avishya matti_avishya  1465 Sep  9 05:13 task15.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 test1.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 test2.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 world.c
matti_avishya@LAPTOP-AUU62HPJ:~/task15$ grep -E "pipe|fork|dup2|execlp|waitpid" task15.c
    /* Create pipe */
    if (pipe(fd) == -1)
        perror("pipe");
    producer = fork();
        perror("fork");
        /* Redirect stdout to pipe */
        dup2(fd[1], STDOUT_FILENO);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("execlp ls");
    consumer = fork();
        perror("fork");
        /* Redirect stdin from pipe */
        dup2(fd[0], STDIN_FILENO);
        execlp("grep", "grep", ".c", (char *)NULL);
```

```
matti_avishya@LAPTOP-AUU62HPJ:~/task15$ touch hello.c world.c notes.txt
matti_avishya@LAPTOP-AUU62HPJ:~/task15$ ./task15
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:14 hello.c
-rw-r--r-- 1 matti_avishya matti_avishya  1465 Sep  9 05:13 task15.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 test1.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 test2.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:14 world.c
Pipeline execution completed.
matti_avishya@LAPTOP-AUU62HPJ:~/task15$ ls -l | grep ".c"
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:14 hello.c
-rw-r--r-- 1 matti_avishya matti_avishya  1465 Sep  9 05:13 task15.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 test1.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 test2.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:14 world.c
matti_avishya@LAPTOP-AUU62HPJ:~/task15$ ls -l task15 task15.c
-rwxr-xr-x 1 matti_avishya matti_avishya 16344 Sep  9 05:14 task15
-rw-r--r-- 1 matti_avishya matti_avishya  1465 Sep  9 05:13 task15.c
matti_avishya@LAPTOP-AUU62HPJ:~/task15$ grep -E "pipe|fork|dup2|execlp|waitpid" task15.c
    /* Create pipe */
    if (pipe(fd) == -1)
        perror("pipe");
    producer = fork();
        perror("fork");
        /* Redirect stdout to pipe */
        dup2(fd[1], STDOUT_FILENO);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("execlp ls");
    consumer = fork();
        perror("fork");
        /* Redirect stdin from pipe */
        dup2(fd[0], STDIN_FILENO);
        execlp("grep", "grep", ".c", (char *)NULL);
        perror("execlp grep");
    /* Parent closes both pipe ends */
    waitpid(producer, NULL, 0);
    waitpid(consumer, NULL, 0);
matti_avishya@LAPTOP-AUU62HPJ:~/task15$ ls -l
total 20
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:14 hello.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:14 notes.txt
-rwxr-xr-x 1 matti_avishya matti_avishya 16344 Sep  9 05:14 task15
-rw-r--r-- 1 matti_avishya matti_avishya  1465 Sep  9 05:13 task15.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 test1.c
-rw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:13 test2.c
```

Task 16

```
matti_avishya@LAPTOP-AUU62HPJ:~/task15$ cd ~
matti_avishya@LAPTOP-AUU62HPJ:~$ mkdir -p task16
matti_avishya@LAPTOP-AUU62HPJ:~$ cd task16
matti_avishya@LAPTOP-AUU62HPJ:~/task16$
matti_avishya@LAPTOP-AUU62HPJ:~/task16$ cat > server.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <signal.h>
#include <string.h>

#define MAX_CLIENTS 3
#define BUFFER_SIZE 256

char c2s[MAX_CLIENTS][50] = {
    "client1_to_server",
    "client2_to_server",
    "client3_to_server"
};

char s2c[MAX_CLIENTS][50] = {
    "server_to_client1",
    "server_to_client2",
    "server_to_client3"
};

pid_t children[MAX_CLIENTS];

void cleanup(int sig)
{
    int i;

    printf("\nServer shutting down...\n");

    for (i = 0; i < MAX_CLIENTS; i++)
    {
        if (children[i] > 0)
            kill(children[i], SIGTERM);
    }
```

```
    while (waitpid(-1, NULL, WNOHANG) > 0)
    {
    }
}

void client_handler(int client_id)
{
    int read_fd;
    int write_fd;
    char buffer[BUFFER_SIZE];
    char response[BUFFER_SIZE];

    read_fd = open(c2s[client_id], O_RDONLY);

    if (read_fd < 0)
    {
        perror("open client-to-server FIFO");
        exit(1);
    }

    write_fd = open(s2c[client_id], O_WRONLY);

    if (write_fd < 0)
    {
        perror("open server-to-client FIFO");
        close(read_fd);
        exit(1);
    }

    printf("Handler %d started. PID = %d\n",
           client_id + 1, getpid());
    fflush(stdout);

    while (1)
    {
        memset(buffer, 0, sizeof(buffer));

        ssize_t n = read(read_fd, buffer, sizeof(buffer) - 1);

        if (n <= 0)
            break;
```

```
            snprintf(response, sizeof(response),
                     "Server: Client %d disconnected.",
                     client_id + 1);

            write(write_fd, response, strlen(response) + 1);
            break;
        }

        printf("Client %d: %s\n", client_id + 1, buffer);
        fflush(stdout);

        snprintf(response, sizeof(response),
                 "Server received from Client %d: %s",
                 client_id + 1, buffer);

        write(write_fd, response, strlen(response) + 1);
    }

    close(read_fd);
    close(write_fd);

    printf("Handler %d exiting.\n", client_id + 1);
    fflush(stdout);

    exit(0);
}

int main()
{
    int i;

    signal(SIGUSR1, handle_usr1);
    signal(SIGINT, cleanup);
    signal(SIGCHLD, handle_chld);

    printf("===== Multi-Client FIFO Server =====\n");

    /* Create all FIFOs */
    for (i = 0; i < MAX_CLIENTS; i++)
    {
        mkfifo(c2s[i], 0666);
        mkfifo(s2c[i], 0666);
    }
```

matti_avishya@LAPTOP-AUU(× Ubuntu

```
    }

    printf("All FIFOs created.\n");

    /* Create one handler for each client */
    for (i = 0; i < MAX_CLIENTS; i++)
    {
        children[i] = fork();

        if (children[i] < 0)
        {
            perror("fork");
            cleanup(SIGINT);
        }

        if (children[i] == 0)
        {
            client_handler(i);
        }
    }

    printf("Server PID: %d\n", getpid());
    printf("Waiting for clients...\n");

    while (1)
    {
        pause();
    }

    return 0;
}
EOF
matti_avishya@LAPTOP-AUU62HPJ:~/task16$
matti_avishya@LAPTOP-AUU62HPJ:~/task16$ cat > client.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <signal.h>
#include <string.h>

#define BUFFER_SIZE 256

char c2s[3][50] = {
```

```c
char s2c[3][50] = {
    "server_to_client1",
    "server_to_client2",
    "server_to_client3"
};

int write_fd;
int read_fd;

void handle_sigusr1(int sig)
{
    printf("\nClient received SIGUSR1.\n");
    fflush(stdout);
}

void handle_sigint(int sig)
{
    printf("\nClient exiting...\n");

    close(write_fd);
    close(read_fd);

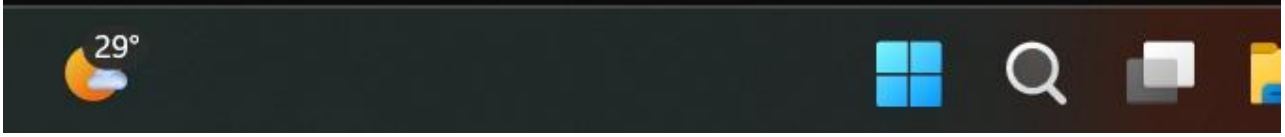
    exit(0);
}

int main(int argc, char *argv[])
{
    int client_id;
    char buffer[BUFFER_SIZE];
    char response[BUFFER_SIZE];

    if (argc != 2)
    {
        printf("Usage: %s <client_number>\n", argv[0]);
        printf("Example: %s 1\n", argv[0]);
        return 1;
    }

    client_id = atoi(argv[1]) - 1;

    if (client_id < 0 || client_id > 2)
    {
        printf("Client number must be 1, 2, or 3.\n");
```



```
signal(SIGUSR1, handle_sigusr1);
signal(SIGINT, handle_sigint);

printf("Client %d started. PID = %d\n",
       client_id + 1, getpid());

/*
 * Open client-to-server FIFO for writing.
 * The server handler opens the read end.
 */
write_fd = open(c2s[client_id], O_WRONLY);

if (write_fd < 0)
{
    perror("open client-to-server FIFO");
    return 1;
}

/*
 * Open server-to-client FIFO for reading.
 */
read_fd = open(s2c[client_id], O_RDONLY);

if (read_fd < 0)
{
    perror("open server-to-client FIFO");
    close(write_fd);
    return 1;
}

printf("Connected to server.\n");

while (1)
{
    printf("Client %d > ", client_id + 1);
    fflush(stdout);

    if (fgets(buffer, sizeof(buffer), stdin) == NULL)
        break;

    buffer[strcspn(buffer, "\n")] = '\0';

    if (strlen(buffer) == 0)
        continue;
```



```
        if (strlen(buffer) == 0)
            continue;

        write(write_fd, buffer, strlen(buffer) + 1);

        memset(response, 0, sizeof(response));

        if (read(read_fd, response, sizeof(response) - 1) <= 0)
            break;

        printf("%s\n", response);

        if (strcmp(buffer, "exit") == 0)
            break;
    }

    close(write_fd);
    close(read_fd);

    printf("Client %d terminated.\n", client_id + 1);

    return 0;
}
EOF
matti_avishya@LAPTOP-AUU62HPJ:~/task16$ gcc -Wall -Wextra server.c -o server
server.c: In function 'cleanup':
server.c:28:18: warning: unused parameter 'sig' [-Wunused-parameter]
   28 | void cleanup(int sig)
      |              ~~~~^~~
server.c: In function 'handle_usr1':
server.c:49:22: warning: unused parameter 'sig' [-Wunused-parameter]
   49 | void handle_usr1(int sig)
      |                  ~~~~^~~
server.c: In function 'handle_chld':
server.c:55:22: warning: unused parameter 'sig' [-Wunused-parameter]
   55 | void handle_chld(int sig)
      |                  ~~~~^~~
server.c: In function 'client_handler':
server.c:115:51: warning: '%s' directive output may be truncated writing up to 255 bytes into a region of size between 215 and 225 [-Wformat-truncation=]
  115 |                 "Server received from Client %d: %s",
      |                                                   ^~
  116 |                 client_id + 1, buffer);
      |                                ~~~~~~
```

```
      |
server.c: In function 'client_handler':
server.c:115:51: warning: '%s' directive output may be truncated writing up to 255 bytes into a r
  115 |                 "Server received from Client %d: %s",
      |                                                   ^~
  116 |                 client_id + 1, buffer);
      |                                ~~~~~~
server.c:114:9: note: 'snprintf' output between 32 and 297 bytes into a destination of size 256
  114 |         snprintf(response, sizeof(response),
      |         ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  115 |                 "Server received from Client %d: %s",
      |                 ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  116 |                 client_id + 1, buffer);
      |                 ~~~~~~~~~~~~~~~~~~~~~~
matti_avishya@LAPTOP-AUU62HPJ:~/task16$ ls -l server
-rwxr-xr-x 1 matti_avishya matti_avishya 17384 Sep  9 05:19 server
matti_avishya@LAPTOP-AUU62HPJ:~/task16$ ./server
===== Multi-Client FIFO Server =====
All FIFOs created.
Server PID: 1144
Waiting for clients...
Handler 1 started. PID = 1145
Client 1: hello
Client 1: cd ~/task16
Client 1: ./client 2
Handler 1 exiting.
Handler 2 started. PID = 1146
Client 2: hello from client 2
Handler 3 started. PID = 1147
Client 3: hello from client 3
Handler 3 exiting.
```

```
matti_avishya@LAPTOP-AUU62HPJ:~$ cd ~/task16
gcc -Wall -Wextra client.c -o client
client.c: In function ‘handle_sigusr1’:
client.c:25:25: warning: unused parameter ‘sig’ [-Wunused-parameter]
   25 | void handle_sigusr1(int sig)
      |                     ~~~~^~~
client.c: In function ‘handle_sigint’:
client.c:31:24: warning: unused parameter ‘sig’ [-Wunused-parameter]
   31 | void handle_sigint(int sig)
      |                    ~~~~^~~
matti_avishya@LAPTOP-AUU62HPJ:~/task16$ ./client 1
Client 1 started. PID = 1205
Connected to server.
Client 1 > hello
Server received from Client 1: hello
Client 1 > cd ~/task16
./clServer received from Client 1: cd ~/task16
Client 1 > ient 2
Server received from Client 1: ./client 2
Client 1 > exit
Server: Client 1 disconnected.
Client 1 terminated.
matti_avishya@LAPTOP-AUU62HPJ:~/task16$ cd ~/task16
./client 2
Client 2 started. PID = 1224
Connected to server.
Client 2 > hello from client 2
Server received from Client 2: hello from client 2
Client 2 >
```

```
Client 3 started. PID = 1264
Connected to server.
Client 3 > hello from client 3
Server received from Client 3: hello from client 3
Client 3 > exit
Server: Client 3 disconnected.
Client 3 terminated.
matti_avishya@LAPTOP-AUU62HPJ:~/task16$ cd ~/task16
ls -l
total 48
-rwxr-xr-x 1 matti_avishya matti_avishya 17296 Sep  9 05:20 client
-rw-r--r-- 1 matti_avishya matti_avishya  2376 Sep  9 05:19 client.c
prw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:22 client1_to_server
prw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:22 client2_to_server
prw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:22 client3_to_server
-rwxr-xr-x 1 matti_avishya matti_avishya 17384 Sep  9 05:19 server
-rw-r--r-- 1 matti_avishya matti_avishya  3238 Sep  9 05:19 server.c
prw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:22 server_to_client1
prw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:22 server_to_client2
prw-r--r-- 1 matti_avishya matti_avishya     0 Sep  9 05:22 server_to_client3
matti_avishya@LAPTOP-AUU62HPJ:~/task16$
```